Propunere de temă de licență: lansator de jocuri multi-platformă

Oct 7, 2026 · @Andrei Dulama

Pe scurt

Am dezvoltat o aplicație desktop pentru Windows care adună într-o singură bibliotecă toate jocurile unui utilizator: din Steam, GOG, Epic, Ubisoft, Xbox (Game Pass) și din foldere, în afara oricărui magazin. Aplicația descoperă singură jocurile pe disc, le afișează cu coperți și detalii, le pornește, măsoară timpul de joc la nivel de proces, ține istoricul sesiunilor, face copii ale salvărilor și arată statistici și spațiul ocupat.

Este scrisă în Python cu PySide6 (Qt): aproximativ 75 de module și 21.000 de linii, organizate pe 7 straturi, cu peste 930 de verificări automate. O folosesc zilnic din august 2026.

Codul a fost generat cu Claude Code, un asistent AI de programare, în 53 de faze, pe baza cerințelor, testelor și deciziilor mele. Parcurg codul modul cu modul și pot explica arhitectura și drumul datelor prin ea.

Problema și motivația pentru un jucător

Un jucător de PC are jocurile în patru-cinci locuri, iar fiecare magazin își vede doar jocurile lui. Nimeni nu îi arată tot ce are într-un singur loc, nu îi măsoară timpul pentru jocurile din afara magazinului, nu îi spune ce a cumpărat și nu a jucat, ce încape într-o seară scurtă, unde îi sunt salvările și ce îi ocupă discul. Aplicația răspunde exact la aceste întrebări.

Din perspectivă economică, jucătorul este un consumator cu buget. Aplicația îi poate da indicatori pe care niciun magazin nu îi oferă: cost pe oră de joc, valoarea jocurilor cumpărate și nejucate (backlog), spațiul ocupat de jocuri neatinse.

Biblioteca pe care s-a testat (a mea, la 7 octombrie 2026):

- 27 de jocuri din 4 surse: 18 în afara magazinelor, 6 Steam, 2 Xbox, 1 GOG; 1,2 TB pe disc.
- 48 de ore cronometrate în 52 de sesiuni pe jocurile instalate; 75 de ore și 109 sesiuni cu jocurile dezinstalate, al căror istoric este păstrat.
- 3 jocuri instalate și niciodată pornite, 58 GB.

- Exemplu de indicator: SILENT HILL Townfall, 49,99 € preț de listă și 10,6 ore, sub 5 €/oră; CONTROL Resonant, 59,99 € și 2,5 ore, 24 €/oră. Prețul este cel de listă din Steam, un indicator de substituție: aplicația nu cunoaște prețul plătit, care ar fi un câmp în baza de date propusă.

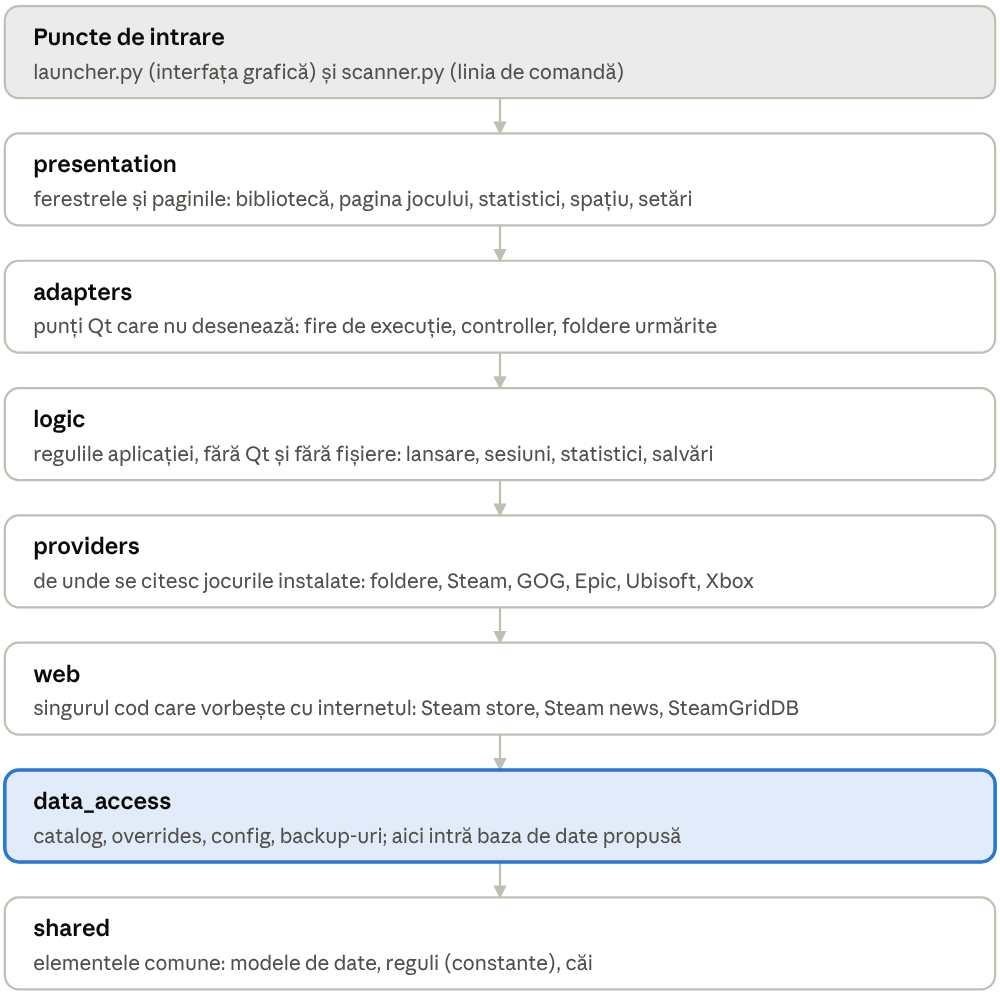
Ce face aplicația astăzi

- Descoperire: șase surse (foldere configurabile, Steam, GOG, Epic, Ubisoft, Xbox); executabilul potrivit este ales după motorul jocului (Unity, Unreal, Godot) sau euristic; duplicatele dintre surse sunt eliminate.
- Bibliotecă: grilă de coperți, căutare, filtre (favorite, în joc, terminate, abandonate); pagina fiecărui joc cu descriere, capturi și noutăți de la Steam, sesiuni recente, notă și stare.
- Timp de joc: măsurat la nivel de proces cu Job Object, ca să prindă și jocul real pornit printr-un lansator intermediar; istoricul sesiunilor; statistici (pe luni, cele mai jucate, surse) și pagina de spațiu pe disc (pe discuri, treemap).
- Siguranța datelor: backup-uri automate cu scriere atomică, copii ale salvărilor după fiecare sesiune și înainte de dezinstalare, restaurare, export și import al bibliotecii.
- Controller și mod TV (DualSense, Xbox), instalator Windows, artwork de la Steam și, opțional, SteamGridDB.

Cum este construită

Python 3.14 și PySide6. Codul este împărțit în șapte straturi, fiecare în pachetul lui, iar dependențele merg doar în jos: un test citește toate importurile și verifică regula la fiecare rulare (layering relaxat: un strat poate sări peste cel de sub el, iar testul numără săriturile).

Dependențele merg doar în jos; baza de date propusă intră în data_access



arhitectura pe straturi · punctele de intrare și cele 7 straturi

Săgețile arată singura direcție permisă a importurilor. Stratul evidențiat este locul unde intră baza de date, fără ca restul programului să afle.

Datele stau în două fișiere JSON cu o singură direcție de scriere: catalog.json, generat la scanare și regenerabil, și overrides.json, cu tot ce a făcut utilizatorul (timp, sesiuni, artă aleasă, note, stare). Scanarea nu scrie niciodată în al doilea. Peste 930 de verificări automate, în 31 de fișiere de test, rulează pe o copie sandbox a datelor. Documentul de arhitectură (docs/ARHITECTURA.md, în română) descrie straturile, diagrama dependențelor, două drumuri prin cod și granița datelor.

Rolul meu: am scris cerințele fiecărei faze, am testat fiecare fază pe biblioteca mea reală, am raportat bug-urile găsite din folosire și am luat deciziile de produs: ce surse se adaugă, ce rămâne în afara aplicației, cum se restructurează codul pe straturi pentru licență.

Propunerea de lucrare de licență

Titlu de lucru: „Sistem informatic pentru gestiunea unei biblioteci de jocuri multi-platformă: arhitectură n-tier, bază de date și modul de recomandare”.

Aplicația existentă ar fi declarată în lucrare ca bază de pornire. Contribuția nouă, scrisă de mine:

1. Baza de date relațională în locul fișierelor JSON. Entități: Joc, Sursă, Sesiune, Gen (relație M:N), Artă, Copie de salvări, Preț plătit, Wishlist. Migrarea datelor existente și rescrierea interogărilor din statistici în SQL. Stratul data_access izolează deja accesul la date, iar documentul de arhitectură listează tot ce atinge azi fișierele JSON din afara acestei granițe, în două variante de cost (cheie-valoare sau schemă normalizată).
2. Modulul de analiză și recomandare, peste baza de date, cu trei criterii:
   - ce să joace utilizatorul din jocurile pe care le are și nu le-a pornit: filtrare bazată pe conținut (content-based filtering) pe metadatele din Steam deja în cache (genuri, categorii, dezvoltator). Pe datele mele, profilul pe ore jucate este 45% Adventure, 44% Action, 8% RPG; din cele 3 jocuri nepornite, The First Berserker: Khazan (Action, Adventure, RPG) obține scorul 0,96, Worship 0,89, Star Wars Episode I: Racer 0,44;
   - ce încape în timpul disponibil, după lungimea mediană a sesiunilor fiecărui joc;
   - experimental, ce merită cumpărat din wishlist după genurile jucate, împreună cu indicatorii economici (cost pe oră, valoarea backlog-ului).
3. Evaluarea pe biblioteca mea și, dacă este posibil, pe bibliotecile câtorva colegi, prin exportul de bibliotecă existent.

Aplicația rămâne desktop pentru că citește registry-ul Windows, pornește și măsoară procese și citește controllerul. Dacă se dorește o componentă web, partea de rapoarte peste baza de date poate primi o interfață web.

Întrebări pentru coordonator

1. Bază de date încorporată (SQLite), care păstrează aplicația instalabilă fără server, sau bază de date server (Oracle, PostgreSQL)?
2. Unde ar trebui să fie centrul de greutate al lucrării: baza de date și rapoartele, sau modulul de recomandare?
3. Cum se declară în lucrare baza de cod generată cu AI (regulamentul universității, art. 22 alin. 3) și cum se formulează declarația de originalitate?

4. Ce titlu preferați și pot depune cererea de coordonare pe numele dumneavoastră până la 18 octombrie 2026?

Termen și contact

Termenul pentru alegerea coordonatorului: Oct 18, 2026.

Dulama Nicolae-Andrei, Informatică economică, anul III, grupa 13LF742, nicolae.dulama@student.unitbv.ro. Codul și documentul de arhitectură pot fi trimise la cerere; depozitul este privat.